

旋转

四元数

欧拉角

<http://motion.pratt.duke.edu/RoboticSystems/3DRotations.html>

轴角到四元数

轴角到旋转矩阵

轴角到欧拉角

旋转矩阵到四元数

旋转矩阵到轴角

旋转矩阵到欧拉角

四元数到轴角

四元数到旋转矩阵

四元数到欧拉角

欧拉角到旋转矩阵

欧拉角到四元数

欧拉角到轴角

完整代码：

轴角到四元数

在四元数小节讲过：

3D旋转公式（四元数型）

任意向量 v 沿着以单位向量定义的旋转轴 u 旋转 θ 度之后的 v' 可以使用四元数乘法来获得

令 $v = [0, \mathbf{v}]$, $q = [\cos(\frac{1}{2}\theta), \sin(\frac{1}{2}\theta)\mathbf{u}]$

则： $v' = qvq^* = qvq^{-1}$

代码：

```
inline Eigen::Quaterniond AxisAngleToQuaternion(const Eigen::AngleAxisd& axis_angle) {
    Eigen::Vector3d axis = axis_angle.axis();
    double angle = axis_angle.angle();
    Eigen::Quaterniond ret;
```

```

ret.w() = std::cos(angle / 2);
ret.x() = std::sin(angle / 2) * axis(0);
ret.y() = std::sin(angle / 2) * axis(1);
ret.z() = std::sin(angle / 2) * axis(2);
return ret;
}

```

轴角到旋转矩阵

我们根据之前讲过的罗德里格斯公式：

罗德里格旋转公式 Rodrigues' Rotation Formula：

3D 空间中任意一个 $\mathbf{v}$ 沿着单位向量 $\mathbf{u}$ 旋转 θ 角度之后的 $\mathbf{v}'$ 为：

$$\mathbf{v}' = \cos(\theta)\mathbf{v} + (1 - \cos(\theta))(\mathbf{u} \cdot \mathbf{v})\mathbf{u} + \sin(\theta)(\mathbf{u} \times \mathbf{v})$$

把等式右边提出一个 $\mathbf{v}$ 出来

$$\begin{aligned}
\mathbf{v}' &= \cos(\theta)\mathbf{v} + (1 - \cos(\theta))(\mathbf{u} \cdot \mathbf{v})\mathbf{u} + \sin(\theta)(\mathbf{u} \times \mathbf{v}) & \hat{\mathbf{u}}\mathbf{v} &= \mathbf{u} \times \mathbf{v} \\
&= \cos(\theta)\mathbf{v} + (1 - \cos(\theta))(\mathbf{u} \cdot \mathbf{v})\mathbf{u} + \sin(\theta)\hat{\mathbf{u}}\mathbf{v} & \text{由拉格朗日有 } \mathbf{u} \times (\mathbf{u} \times \mathbf{v}) &= (\mathbf{u} \cdot \mathbf{v})\mathbf{u} - \mathbf{v} \\
&= \cos(\theta)\mathbf{v} + \sin(\theta)\hat{\mathbf{u}}\mathbf{v} + (1 - \cos(\theta))(I + \hat{\mathbf{u}}^2)\mathbf{v} \\
&= I\mathbf{v} + \sin(\theta)\hat{\mathbf{u}}\mathbf{v} + (1 - \cos(\theta))\hat{\mathbf{u}}^2\mathbf{v} \\
&= (I + \sin(\theta)\hat{\mathbf{u}} + (1 - \cos(\theta))\hat{\mathbf{u}}^2)\mathbf{v}
\end{aligned}$$

可得：

$$R = I + \sin(\theta)\hat{\mathbf{u}} + (1 - \cos(\theta))\hat{\mathbf{u}}^2$$

其中的

$$\hat{\mathbf{u}} = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}$$

$$\hat{\mathbf{u}}^2 = \begin{bmatrix} -u_z^2 - u_y^2 & u_x u_y & u_x u_z \\ u_x u_y & -u_x^2 - u_y^2 & u_y u_z \\ u_x u_z & u_y u_z & -u_x^2 - u_y^2 \end{bmatrix}$$

代码：

```

inline Eigen::Matrix3d AxisAngleToRotationMatrix(const Eigen::AngleAxisd& axis_angle) {
    Eigen::Vector3d axis = axis_angle.axis();
    double angle = axis_angle.angle();
    Eigen::Matrix3d ret;
    ret = Eigen::Matrix3d::Identity() + std::sin(angle) * VectorHat(axis) +
        (1 - std::cos(angle)) * VectorHat(axis) * VectorHat(axis);
}

```

```
return ret;
}
```

轴角到欧拉角

这个是间接实现的

先从轴角到旋转矩阵，再从旋转矩阵到欧拉角

代码：

```
inline Eigen::Vector3d AxisAngleToEuler(const Eigen::AngleAxisd& axis_angle) {
    return gxt::RotationMatrixToEuler(gxt::AxisAngleToRotationMatrix(axis_angle));
}
```

旋转矩阵到四元数

同样根据之前推导的四元数和旋转矩阵的关系：

3D旋转公式（矩阵型）

任意向量 $\mathbf{v}$ 沿着以单位向量定义的旋转轴 $\mathbf{u}$ 旋转 θ 角度之后的 $\mathbf{v}'$ 可以使用矩阵乘法来获得

令 $a = \cos(\frac{1}{2}\theta)$, $b = \sin(\frac{1}{2}\theta)u_x$, $c = \sin(\frac{1}{2}\theta)u_y$, $d = \sin(\frac{1}{2}\theta)u_z$

那么：

$$\mathbf{v}' = \begin{bmatrix} 1 - 2c^2 - 2d^2 & 2bc - 2ad & 2ac + 2bd \\ 2bc + 2ad & 1 - 2b^2 - 2d^2 & 2cd - 2ab \\ 2bd - 2ac & 2ab + 2cd & 1 - 2b^2 - 2c^2 \end{bmatrix} \mathbf{v}$$

将对角线相加得到的就是 $3 - 4(b^2 + c^2 + d^2)$

再根据 $a^2 + b^2 + c^2 + d^2 = 1$ 可以轻易得到 a

再有： $R_{21} - R_{12} = 4ad$ 可以得到 d

同理可以知道 b 和 c

但实际上因为平方的存在，会有正负号的不同结果，判断就会更加复杂一些，可以参考：

<https://math.stackexchange.com/questions/893984/conversion-of-rotation-matrix-to-quaternion>

<https://d3cw3dd2w32x2b.cloudfront.net/wp-content/uploads/2015/01/matrix-to-quat.pdf>

代码：

```
inline Eigen::Quaterniond RotationMatrixToQuaternion(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::Quaterniond ret;
    double tr_R=rotation_matrix(0,0)+rotation_matrix(1,1)+rotation_matrix(2,2);
    ret.w()=std::sqrt((1+tr_R)/4);
```

```

ret.x()=(rotation_matrix(2,1)-rotation_matrix(1,2))/(4*ret.w());
ret.y()=(rotation_matrix(0,2)-rotation_matrix(2,0))/(4*ret.w());
ret.z()=(rotation_matrix(1,0)-rotation_matrix(0,1))/(4*ret.w());
return ret;
}

```

上面实现的是一个简略版本，实际可能有正负号问题，对此可以参考上面的链接的pdf的代码：

```

if (m22 < 0) {
    if (m00 > m11) {
        t = 1 + m00 - m11 - m22;
        q = Quaternion( t, m01+m10, m20+m02, m12-m21 );
    }
    else {
        t = 1 - m00 + m11 - m22;
        q = Quaternion( m01+m10, t, m12+m21, m20-m02 );
    }
}
else {
    if (m00 < -m11) {
        t = 1 - m00 - m11 + m22;
        q = Quaternion( m20+m02, m12+m21, t, m01-m10 );
    }
    else {
        t = 1 + m00 + m11 + m22;
        q = Quaternion( m12-m21, m20-m02, m01-m10, t );
    }
}
q *= 0.5 / Sqrt(t);

```

旋转矩阵到轴角

由之前的轴角到旋转矩阵，我们有了： $R = I + \sin(\theta)\hat{\mathbf{u}} + (1 - \cos(\theta))\hat{\mathbf{u}}^2$

现在我们根据这个 R ，推出轴角的 $\mathbf{u}$ 和 θ

首先对两边都取迹：

$$\begin{aligned}
 \text{tr}(R) &= 3 + (1 - \cos(\theta))\text{tr}(\hat{\mathbf{u}}^2) \\
 &= 3 + (1 - \cos(\theta))(-\|\mathbf{u}\|^2) \\
 &= 1 + 2\cos(\theta)
 \end{aligned}$$

$$\text{所以：}\theta = \arccos\left(\frac{\text{tr}(R)-1}{2}\right)$$

然后利用 R 的非对角线元素求解 $\mathbf{u}$ ：

$$\begin{aligned}
 r_{21} - r_{12} &= 2u_z \sin(\theta) \\
 r_{13} - r_{31} &= 2u_y \sin(\theta)
 \end{aligned}$$

$$r_{32} - r_{23} = u_x \sin(\theta)$$

由此：

$$u_x = (r_{32} - r_{23}) / 2 \sin(\theta)$$

$$u_y = (r_{13} - r_{31}) / 2 \sin(\theta)$$

$$u_z = (r_{21} - r_{12}) / 2 \sin(\theta)$$

所以：

$$\mathbf{u} = \left[(r_{32} - r_{23}) / 2 \sin(\theta) \quad (r_{13} - r_{31}) / 2 \sin(\theta) \quad (r_{21} - r_{12}) / 2 \sin(\theta) \right]^T$$

代码：

```
inline Eigen::AngleAxisd RotationMatrixToAxisAngle(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::AngleAxisd ret;
    double tr_R = rotation_matrix(0,0) + rotation_matrix(1,1) + rotation_matrix(2,2);
    ret.angle() = std::acos((tr_R - 1) / 2);
    ret.axis()(0) = (rotation_matrix(2, 1) - rotation_matrix(1, 2)) / (2 * std::sin(ret.angle()));
    ret.axis()(1) = (rotation_matrix(0, 2) - rotation_matrix(2, 0)) / (2 * std::sin(ret.angle()));
    ret.axis()(2) = (rotation_matrix(1, 0) - rotation_matrix(0, 1)) / (2 * std::sin(ret.angle()));
    return ret;
}
```

旋转矩阵到欧拉角

根据欧拉角的旋转矩阵可以进行计算

$$R = R_z(\phi) R_y(\theta) R_x(\psi)$$

$$= \begin{bmatrix} \cos\theta\cos\phi & \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi & \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi \\ \cos\theta\sin\phi & \sin\psi\sin\theta\sin\phi + \cos\psi\cos\phi & \cos\psi\sin\theta\sin\phi - \sin\psi\cos\phi \\ -\sin\theta & \sin\psi\cos\theta & \cos\psi\cos\theta \end{bmatrix}$$

可以根据值的特点计算得到：

$$\frac{R_{32}}{R_{33}} = \tan(\psi)$$

$$\text{所以 } \psi = \arctan\left(\frac{R_{32}}{R_{33}}\right)$$

$$\frac{R_{21}}{R_{11}} = \tan(\phi)$$

$$\text{所以 } \phi = \arctan\left(\frac{R_{21}}{R_{11}}\right)$$

$$R_{31} = -\sin\theta$$

$$\text{所以 } \theta = \arcsin(-R_{31})$$

在实际中，因为数值范围和被除数为0的可能，需要做更严格的判断，具体可以看

<https://eecs.qmul.ac.uk/~gslabaugh/publications/euler.pdf>

代码：

```

inline Eigen::Vector3d RotationMatrixToEuler(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::Vector3d ret;
    ret(0)=std::atan2(rotation_matrix(2,1),rotation_matrix(2,2));
    ret(1)=std::asin(-rotation_matrix(2,0));
    ret(2)=std::atan2(rotation_matrix(1,0),rotation_matrix(0,0));
    return ret;
}

```

上面的代码是属于极致简化版本，没有考虑到很多情况。

比较全的完整版可以参考 [Eigen](#) 的函数实现：这里的输入在 [外旋定轴RPY](#) 的情况是 [\(2,1,0\)](#)

```

template<typename Derived>
EIGEN_DEVICE_FUNC inline Matrix<typename MatrixBase<Derived>::Scalar,3,1>
MatrixBase<Derived>::eulerAngles(Index a0, Index a1, Index a2) const
{
    EIGEN_USING_STD(atan2)
    EIGEN_USING_STD(sin)
    EIGEN_USING_STD(cos)
    /* Implemented from Graphics Gems IV */
    EIGEN_STATIC_ASSERT_MATRIX_SPECIFIC_SIZE(Derived,3,3)

    Matrix<Scalar,3,1> res;
    typedef Matrix<typename Derived::Scalar,2,1> Vector2;

    const Index odd = ((a0+1)%3 == a1) ? 0 : 1;
    const Index i = a0;
    const Index j = (a0 + 1 + odd)%3;
    const Index k = (a0 + 2 - odd)%3;

    if (a0==a2)
    {
        res[0] = atan2(coeff(j,i), coeff(k,i));
        if((odd && res[0]<Scalar(0)) || ((!odd) && res[0]>Scalar(0)))
        {
            {
                if(res[0] > Scalar(0)) {
                    res[0] -= Scalar(EIGEN_PI);
                }
                else {
                    res[0] += Scalar(EIGEN_PI);
                }
            }
            Scalar s2 = Vector2(coeff(j,i), coeff(k,i)).norm();
            res[1] = -atan2(s2, coeff(i,i));
        }
    }
    else

```

```

{
    Scalar s2 = Vector2(coeff(j,i), coeff(k,i)).norm();
    res[1] = atan2(s2, coeff(i,i));
}

// With a=(0,1,0), we have i=0; j=1; k=2, and after computing the first two angles,
// we can compute their respective rotation, and apply its inverse to M. Since the result must
// be a rotation around x, we have:
//
// c2 s1.s2 c1.s2      1 0 0
// 0 c1 -s1 * M = 0 c3 s3
// -s2 s1.c2 c1.c2      0 -s3 c3
//
// Thus: m11.c1 - m21.s1 = c3 & m12.c1 - m22.s1 = s3

Scalar s1 = sin(res[0]);
Scalar c1 = cos(res[0]);
res[2] = atan2(c1*coeff(j,k)-s1*coeff(k,k), c1*coeff(j,j) - s1 * coeff(k,j));
}
else
{
    res[0] = atan2(coeff(j,k), coeff(k,k));
    Scalar c2 = Vector2(coeff(i,i), coeff(i,j)).norm();
    if((odd && res[0]<Scalar(0)) || (!odd && res[0]>Scalar(0))) {
        if(res[0] > Scalar(0)) {
            res[0] -= Scalar(EIGEN_PI);
        }
        else {
            res[0] += Scalar(EIGEN_PI);
        }
    }
    res[1] = atan2(-coeff(i,k), -c2);
}
else
    res[1] = atan2(-coeff(i,k), c2);
Scalar s1 = sin(res[0]);
Scalar c1 = cos(res[0]);
res[2] = atan2(s1*coeff(k,i)-c1*coeff(j,i), c1*coeff(j,j) - s1 * coeff(k,j));
}
if (!odd)
    res = -res;

return res;
}

```

四元数到轴角

这个之前在四元数部分已经推导过了

3D旋转公式（四元数型）

任意向量 v 沿着以单位向量定义的旋转轴 u 旋转 θ 度之后的 v' 可以使用四元数乘法来获得

令 $v = [0, \mathbf{v}]$, $q = [\cos(\frac{1}{2}\theta), \sin(\frac{1}{2}\theta)\mathbf{u}]$

则： $v' = qvq^* = qvq^{-1}$

代码：

```

inline Eigen::AngleAxisd QuaternionToAxisAngle(const Eigen::Quaterniond& quaternion) {
    Eigen::AngleAxisd ret;
    double a=quaternion.w();
    double b=quaternion.x();
    double c=quaternion.y();
    double d=quaternion.z();
    double angle=2*std::acos(a);
    ret.angle()=angle;
    double axis_x=b/std::sin(angle/2);
    double axis_y=c/std::sin(angle/2);
    double axis_z=d/std::sin(angle/2);
    ret.axis()(0)=axis_x;
    ret.axis()(1)=axis_y;
    ret.axis()(2)=axis_z;
    return ret;
}

```

四元数到旋转矩阵

这个之前在四元数部分已经推导过了

3D旋转公式（矩阵型）

任意向量 v 沿着以单位向量定义的旋转轴 u 旋转 θ 角度之后的 v' 可以使用矩阵乘法来获得

令 $a = \cos(\frac{1}{2}\theta)$, $b = \sin(\frac{1}{2}\theta)u_x$, $c = \sin(\frac{1}{2}\theta)u_y$, $d = \sin(\frac{1}{2}\theta)u_z$

那么：

$$v' = \begin{bmatrix} 1 - 2c^2 - 2d^2 & 2bc - 2ad & 2ac + 2bd \\ 2bc + 2ad & 1 - 2b^2 - 2d^2 & 2cd - 2ab \\ 2bd - 2ac & 2ab + 2cd & 1 - 2b^2 - 2c^2 \end{bmatrix} v$$

代码：

```
inline Eigen::Matrix3d QuaternionToRotationMatrix(const Eigen::Quaterniond& quaternion) {
    Eigen::Matrix3d ret=Eigen::Matrix3d::Zero();
    double a=quaternion.w();
    double b=quaternion.x();
    double c=quaternion.y();
    double d=quaternion.z();
    ret <<
        1-2*c*c-2*d*d, 2*b*c-2*a*d, 2*a*c+2*b*d,
        2*b*c+2*a*d, 1-2*b*b-2*d*d, 2*c*d-2*a*b,
        2*b*d-2*a*c, 2*a*b+2*c*d, 1-2*b*b-2*c*c;
    return ret;
}
```

四元数到欧拉角

我们先来推导欧拉角和四元数之间的关系：也就是有了欧拉角怎么得到四元数

先把欧拉角分成三次轴角的旋转，然后再分别对每次轴角变成四元数，最后把三个四元数乘起来就是最终的四元数

可以参考：https://en.wikipedia.org/wiki/Conversion_between_quaternions_and_Euler_angles

$$q_{euler} = \begin{bmatrix} \cos(\psi/2) \\ 0 \\ 0 \\ \sin(\psi/2) \end{bmatrix} \begin{bmatrix} \cos(\theta/2) \\ 0 \\ \sin(\theta/2) \\ 0 \end{bmatrix} \begin{bmatrix} \cos(\phi/2) \\ \sin(\phi/2) \\ 0 \\ 0 \end{bmatrix} \quad \text{四元数连乘展开}$$
$$= \begin{bmatrix} \cos(\phi/2)\cos(\theta/2)\cos(\psi/2) + \sin(\phi/2)\sin(\theta/2)\sin(\psi/2) \\ \sin(\phi/2)\cos(\theta/2)\cos(\psi/2) - \cos(\phi/2)\sin(\theta/2)\sin(\psi/2) \\ \cos(\phi/2)\sin(\theta/2)\cos(\psi/2) + \sin(\phi/2)\cos(\theta/2)\sin(\psi/2) \\ \cos(\phi/2)\cos(\theta/2)\sin(\psi/2) - \sin(\phi/2)\sin(\theta/2)\cos(\psi/2) \end{bmatrix}$$

然后我们反解这四个方程得到的就是欧拉角

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} \text{atan2}(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)) \\ -\pi/2 + 2\text{atan2}(\sqrt{1 + 2(q_w q_y - q_x q_z)}, \sqrt{1 - 2(q_w q_y - q_x q_z)}) \\ \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \end{bmatrix}$$

代码：

```
inline Eigen::Vector3d QuaternionToEuler(const Eigen::Quaterniond& quaternion) {
    Eigen::Vector3d ret;
    double w=quaternion.w();
    double x=quaternion.x();
    double y=quaternion.y();
```

```
double z=quaternion.z();

ret(0)=std::atan2(2*(w*x+y*z),1-2*(x*x+y*y));
ret(1)=-M_PI/2+2*std::atan2(std::sqrt(1+2*(w*y-x*z)),std::sqrt(1-2*(w*y-x*z)));
ret(2)=std::atan2(2*(w*z+x*y),1-2*(y*y+z*z));
return ret;
}
```

因为涉及到大量矩阵运算，效率低，我们可以考虑间接实现

先从四元数到旋转矩阵，再从旋转矩阵到欧拉角

```
inline Eigen::Vector3d QuaternionToEuler(const Eigen::Quaterniond& quaternion) {
    return gxt::RotationMatrixToEuler(gxt::QuaternionToRotationMatrix(quaternion));
}
```

欧拉角到旋转矩阵

欧拉角到旋转矩阵很基础，已经推导过，其实就是：

$$R = R_z(\phi)R_y(\theta)R_x(\psi)$$

$$= \begin{bmatrix} \cos\theta\cos\phi & \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi & \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi \\ \cos\theta\sin\phi & \sin\psi\sin\theta\sin\phi + \cos\psi\cos\phi & \cos\psi\sin\theta\sin\phi - \sin\psi\cos\phi \\ -\sin\theta & \sin\psi\cos\theta & \cos\psi\cos\theta \end{bmatrix}$$

代码：

```
inline Eigen::Matrix3d EulerToRotationMatrix(const Eigen::Vector3d& euler) {
    Eigen::Matrix3d ret=Eigen::Matrix3d::Zero();
    Eigen::Matrix3d R_x=Eigen::Matrix3d::Zero();
    Eigen::Matrix3d R_y=Eigen::Matrix3d::Zero();
    Eigen::Matrix3d R_z=Eigen::Matrix3d::Zero();

    double x=euler(0);
    double y=euler(1);
    double z=euler(2);

    R_x(0,0)=1;
    R_x(1,1)=std::cos(x);
    R_x(1,2)=-std::sin(x);
    R_x(2,1)=std::sin(x);
    R_x(2,2)=std::cos(x);

    R_y(0,0)=std::cos(y);
    R_y(0,2)=std::sin(y);
    R_y(1,1)=1;
    R_y(2,1)=std::sin(y);
    R_y(2,2)=std::cos(y);

    R_z(0,0)=std::cos(z);
    R_z(0,1)=std::sin(z);
    R_z(1,0)=-std::sin(z);
    R_z(1,1)=std::cos(z);
    R_z(2,0)=0;
    R_z(2,1)=0;
    R_z(2,2)=1;

    ret=R_z*R_y*R_x;
    return ret;
}
```

```

R_y(2,0)=-std::sin(y);
R_y(2,2)=std::cos(y);

R_z(0,0)=std::cos(z);
R_z(0,1)=-std::sin(z);
R_z(1,0)=std::sin(z);
R_z(1,1)=std::cos(z);
R_z(2,2)=1;

ret=R_z*R_y*R_x;
return ret;
}

```

欧拉角到四元数

之前已经讲过欧拉角和四元数的关系

$$\begin{aligned}
 q_{euler} &= \begin{bmatrix} \cos(\psi/2) \\ 0 \\ 0 \\ \sin(\psi/2) \end{bmatrix} \begin{bmatrix} \cos(\theta/2) \\ 0 \\ \sin(\theta/2) \\ 0 \end{bmatrix} \begin{bmatrix} \cos(\phi/2) \\ \sin(\phi/2) \\ 0 \\ 0 \end{bmatrix} \quad \text{四元数连乘展开} \\
 &= \begin{bmatrix} \cos(\phi/2)\cos(\theta/2)\cos(\psi/2) + \sin(\phi/2)\sin(\theta/2)\sin(\psi/2) \\ \sin(\phi/2)\cos(\theta/2)\cos(\psi/2) - \cos(\phi/2)\sin(\theta/2)\sin(\psi/2) \\ \cos(\phi/2)\sin(\theta/2)\cos(\psi/2) + \sin(\phi/2)\cos(\theta/2)\sin(\psi/2) \\ \cos(\phi/2)\cos(\theta/2)\sin(\psi/2) - \sin(\phi/2)\sin(\theta/2)\cos(\psi/2) \end{bmatrix}
 \end{aligned}$$

这里我们直接带入右边的参数就得到四元数了

代码：

```

inline Eigen::Quaterniond EulerToQuaternion(const Eigen::Vector3d& euler) {
    Eigen::Quaterniond quaternion;
    double x=euler(0);
    double y=euler(1);
    double z=euler(2);

    double cx=std::cos(x/2); double cy=std::cos(y/2); double cz=std::cos(z/2);
    double sx=std::sin(x/2); double sy=std::sin(y/2); double sz=std::sin(z/2);
    quaternion.w()=cx*cy*cz+sx*sy*sz;
    quaternion.x()=sx*cy*cz-cx*sy*sz;
    quaternion.y()=cx*sy*cz+sx*cy*sz;
    quaternion.z()=cx*cy*sz-sx*sy*cz;

    return quaternion;
}

```

欧拉角到轴角

这里我们间接实现

先从欧拉角到四元数，再从四元数到轴角

代码：

```
inline Eigen::AngleAxisd EulerToAxisAngle(const Eigen::Vector3d& euler) {  
    return gxt::QuaternionToAxisAngle(gxt::EulerToQuaternion(euler));  
}
```

完整代码：

▼ rotation_convert.hpp

```
#pragma once  
  
#include <Eigen/Core>  
#include <Eigen/Dense>  
#include <cmath>  
  
#include "common.h"  
  
namespace Eigen {  
  
    // clang-format off  
  
    inline Eigen::Matrix3d AxisAngleToRotationMatrix(const Eigen::AngleAxisd& axis_angle) {  
        Eigen::AngleAxisd rotation(axis_angle);  
        return rotation.toRotationMatrix();  
    }  
    inline Eigen::Matrix3d AxisAngleToRotationMatrix(const Eigen::Vector3d& axis, double angle) {  
        Eigen::AngleAxisd rotation(angle, axis);  
        return rotation.toRotationMatrix();  
    }  
    inline Eigen::Vector3d AxisAngleToEuler(const Eigen::AngleAxisd& axis_angle) {  
        Eigen::Matrix3d rotation_matrix = axis_angle.toRotationMatrix();  
        Eigen::Vector3d euler_angles = rotation_matrix.eulerAngles(2, 1, 0);  
        return euler_angles;  
    }  
    inline Eigen::Vector3d AxisAngleToEuler(const Eigen::Vector3d& axis, double angle) {
```

```

Eigen::AngleAxisd axis_angle(angle, axis);
Eigen::Matrix3d rotation_matrix = axis_angle.toRotationMatrix();
Eigen::Vector3d euler_angles = rotation_matrix.eulerAngles(2, 1, 0);
return euler_angles;
}

inline Eigen::Quaterniond AxisAngleToQuaternion(const Eigen::AngleAxisd& axis_angle) {
    return Eigen::Quaterniond(axis_angle);
}

inline Eigen::Quaterniond AxisAngleToQuaternion(const Eigen::Vector3d& axis, double angle) {
    Eigen::AngleAxisd rotation(angle, axis);
    return Eigen::Quaterniond(rotation);
}

inline Eigen::Vector3d RotationMatrixToEuler(const Eigen::Matrix3d& rotationMatrix) {
    return rotationMatrix.eulerAngles(2, 1, 0);
}

inline Eigen::Quaterniond RotationMatrixToQuaternion(const Eigen::Matrix3d& rotationMatrix) {
    return Eigen::Quaterniond(rotationMatrix);
}

inline Eigen::AngleAxisd RotationMatrixToAxisAngle(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::AngleAxisd axis_angle(rotation_matrix);
    return axis_angle;
}

inline Eigen::Matrix3d QuaternionToRotationMatrix(const Eigen::Quaterniond& quaternion) {
    return quaternion.toRotationMatrix();
}

inline Eigen::Vector3d QuaternionToEuler(const Eigen::Quaterniond& quaternion) {
    return quaternion.toRotationMatrix().eulerAngles(2, 1, 0);
}

inline Eigen::AngleAxisd QuaternionToAxisAngle(const Eigen::Quaterniond& quaternion) {
    return Eigen::AngleAxisd(quaternion);
}

inline Eigen::Quaterniond EulerToQuaternion(const Eigen::Vector3d& euler) {
    Eigen::Quaterniond quaternion;
    quaternion = Eigen::AngleAxisd(euler(2), Eigen::Vector3d::UnitZ()) *
        Eigen::AngleAxisd(euler(1), Eigen::Vector3d::UnitY()) *
        Eigen::AngleAxisd(euler(0), Eigen::Vector3d::UnitX());
    return quaternion;
}

inline Eigen::Matrix3d EulerToRotationMatrix(const Eigen::Vector3d& euler) {
    return EulerToQuaternion(euler).toRotationMatrix();
}

```

```

}
inline Eigen::AngleAxisd EulerToAxisAngle(const Eigen::Vector3d& euler) {
    return Eigen::AngleAxisd(EulerToQuaternion(euler));
}

// clang-format on

} // namespace Eigen

// clang-format off
namespace gxt {

namespace {
inline Eigen::Matrix3d VectorHat(const Eigen::Vector3d& vector) {
    Eigen::Matrix3d matrix=Eigen::Matrix3d::Zero();
    matrix(0,1)=-vector(2);
    matrix(1,0)=vector(2);
    matrix(0,2)=vector(1);
    matrix(2,0)=-vector(1);
    matrix(1,2)=-vector(0);
    matrix(2,1)=vector(0);
    return matrix;
}
}

inline Eigen::Vector3d RotationMatrixToEuler(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::Vector3d ret;
    ret(0)=std::atan2(rotation_matrix(2,1),rotation_matrix(2,2));
    ret(1)=std::asin(-rotation_matrix(2,0));
    ret(2)=std::atan2(rotation_matrix(1,0),rotation_matrix(0,0));
    return ret;
}

inline Eigen::Matrix3d AxisAngleToRotationMatrix(const Eigen::AngleAxisd& axis_angle) {
    Eigen::Vector3d axis = axis_angle.axis();
    double angle = axis_angle.angle();
    Eigen::Matrix3d ret;
    ret = Eigen::Matrix3d::Identity() + std::sin(angle) * VectorHat(axis) +
        (1 - std::cos(angle)) * VectorHat(axis) * VectorHat(axis);
    return ret;
}

inline Eigen::Matrix3d AxisAngleToRotationMatrix(const Eigen::Vector3d& axis, double angle) {
    Eigen::Matrix3d ret;
    ret = Eigen::Matrix3d::Identity() + std::sin(angle) * VectorHat(axis) +
        (1 - std::cos(angle)) * VectorHat(axis) * VectorHat(axis);
}

```

```

    return ret;
}
inline Eigen::Quaterniond AxisAngleToQuaternion(const Eigen::AngleAxisd& axis_angle) {
    Eigen::Vector3d axis = axis_angle.axis();
    double angle = axis_angle.angle();
    Eigen::Quaterniond ret;
    ret.w() = std::cos(angle / 2);
    ret.x() = std::sin(angle / 2) * axis(0);
    ret.y() = std::sin(angle / 2) * axis(1);
    ret.z() = std::sin(angle / 2) * axis(2);
    return ret;
}
inline Eigen::Quaterniond AxisAngleToQuaternion(const Eigen::Vector3d& axis, double angle) {
    Eigen::Quaterniond ret;
    ret.w() = std::cos(angle / 2);
    ret.x() = std::sin(angle / 2) * axis(0);
    ret.y() = std::sin(angle / 2) * axis(1);
    ret.z() = std::sin(angle / 2) * axis(2);
    return ret;
}
inline Eigen::Vector3d AxisAngleToEuler(const Eigen::AngleAxisd& axis_angle) {
    return gxt::RotationMatrixToEuler(gxt::AxisAngleToRotationMatrix(axis_angle));
}
inline Eigen::Vector3d AxisAngleToEuler(const Eigen::Vector3d& axis, double angle) {
    return gxt::RotationMatrixToEuler(gxt::AxisAngleToRotationMatrix(axis, angle));
}

inline Eigen::AngleAxisd RotationMatrixToAxisAngle(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::AngleAxisd ret;
    double tr_R = rotation_matrix(0,0) + rotation_matrix(1,1) + rotation_matrix(2,2);
    ret.angle() = std::acos((tr_R - 1) / 2);
    ret.axis()(0) = (rotation_matrix(2, 1) - rotation_matrix(1, 2)) / (2 * std::sin(ret.angle()));
    ret.axis()(1) = (rotation_matrix(0, 2) - rotation_matrix(2, 0)) / (2 * std::sin(ret.angle()));
    ret.axis()(2) = (rotation_matrix(1, 0) - rotation_matrix(0, 1)) / (2 * std::sin(ret.angle()));
    return ret;
}
inline Eigen::Quaterniond RotationMatrixToQuaternion(const Eigen::Matrix3d& rotation_matrix) {
    Eigen::Quaterniond ret;
    double tr_R = rotation_matrix(0,0) + rotation_matrix(1,1) + rotation_matrix(2,2);
    ret.w() = std::sqrt((1 + tr_R) / 4);
    ret.x() = (rotation_matrix(2,1) - rotation_matrix(1,2)) / (4 * ret.w());
    ret.y() = (rotation_matrix(0,2) - rotation_matrix(2,0)) / (4 * ret.w());
    ret.z() = (rotation_matrix(1,0) - rotation_matrix(0,1)) / (4 * ret.w());
    return ret;
}

```

```

}

inline Eigen::Matrix3d QuaternionToRotationMatrix(const Eigen::Quaterniond& quaternion) {
    Eigen::Matrix3d ret=Eigen::Matrix3d::Zero();
    double a=quaternion.w();
    double b=quaternion.x();
    double c=quaternion.y();
    double d=quaternion.z();
    ret <<
        1-2*c*c-2*d*d, 2*b*c-2*a*d, 2*a*c+2*b*d,
        2*b*c+2*a*d, 1-2*b*b-2*d*d, 2*c*d-2*a*b,
        2*b*d-2*a*c, 2*a*b+2*c*d, 1-2*b*b-2*c*c ;
    return ret;
}

inline Eigen::AngleAxisd QuaternionToAxisAngle(const Eigen::Quaterniond& quaternion) {
    Eigen::AngleAxisd ret;
    double a=quaternion.w();
    double b=quaternion.x();
    double c=quaternion.y();
    double d=quaternion.z();
    double angle=2*std::acos(a);
    ret.angle()=angle;
    double axis_x=b/std::sin(angle/2);
    double axis_y=c/std::sin(angle/2);
    double axis_z=d/std::sin(angle/2);
    ret.axis()(0)=axis_x;
    ret.axis()(1)=axis_y;
    ret.axis()(2)=axis_z;
    return ret;
}

inline Eigen::Vector3d QuaternionToEuler(const Eigen::Quaterniond& quaternion) {
    Eigen::Vector3d ret;
    double w=quaternion.w();
    double x=quaternion.x();
    double y=quaternion.y();
    double z=quaternion.z();

    ret(0)=std::atan2(2*(w*x+y*z),1-2*(x*x+y*y));
    ret(1)=-M_PI/2+2*std::atan2(std::sqrt(1+2*(w*y-x*z)),std::sqrt(1-2*(w*y-x*z)));
    ret(2)=std::atan2(2*(w*z+x*y),1-2*(y*y+z*z));
    return ret;
}

inline Eigen::Matrix3d EulerToRotationMatrix(const Eigen::Vector3d& euler) {

```



```

Eigen::Matrix3d ret=Eigen::Matrix3d::Zero();
Eigen::Matrix3d R_x=Eigen::Matrix3d::Zero();
Eigen::Matrix3d R_y=Eigen::Matrix3d::Zero();
Eigen::Matrix3d R_z=Eigen::Matrix3d::Zero();

double x=euler(0);
double y=euler(1);
double z=euler(2);

R_x(0,0)=1;
R_x(1,1)=std::cos(x);
R_x(1,2)=-std::sin(x);
R_x(2,1)=std::sin(x);
R_x(2,2)=std::cos(x);

R_y(0,0)=std::cos(y);
R_y(0,2)=std::sin(y);
R_y(1,1)=1;
R_y(2,0)=-std::sin(y);
R_y(2,2)=std::cos(y);

R_z(0,0)=std::cos(z);
R_z(0,1)=-std::sin(z);
R_z(1,0)=std::sin(z);
R_z(1,1)=std::cos(z);
R_z(2,2)=1;

ret=R_z*R_y*R_x;
return ret;
}

inline Eigen::Quaterniond EulerToQuaternion(const Eigen::Vector3d& euler) {
    Eigen::Quaterniond quaternion;
    double x=euler(0);
    double y=euler(1);
    double z=euler(2);

    double cx=std::cos(x/2); double cy=std::cos(y/2); double cz=std::cos(z/2);
    double sx=std::sin(x/2); double sy=std::sin(y/2); double sz=std::sin(z/2);
    quaternion.w()=cx*cy*cz+sx*sy*sz;
    quaternion.x()=sx*cy*cz-cx*sy*sz;
    quaternion.y()=cx*sy*cz+sx*cy*sz;
    quaternion.z()=cx*cy*sz-sx*sy*cz;

    return quaternion;
}

```

```

inline Eigen::AngleAxisd EulerToAxisAngle(const Eigen::Vector3d& euler) {
    return gxt::QuaternionToAxisAngle(gxt::EulerToQuaternion(euler));
}

} // namespace gxt

// clang-format on

```

▼ rotation_convert.cc

```

#include "common.h"
#include "rotation_convert.hpp"

// clang-format off

// Axis Angle
Eigen::Vector3d axis_temp_(1, 2, 3);
Eigen::Vector3d axis=axis_temp_.normalized();
double angle = M_PI / 2;
Eigen::AngleAxisd axis_angle(angle, axis);

// Rotation Matrix
Eigen::Matrix3d rotation_matrix = (Eigen::Matrix3d() <<
    0.0714286, -0.658927, 0.748808,
    0.944641, 0.285714, 0.16131,
    -0.320237, 0.695833, 0.642857
).finished();

// Quaternion
Eigen::Quaterniond quaternion(0.707107, 0.188982, 0.377964, 0.566947);

// Euler Angle
// 我们定义的欧拉角顺序是外旋定轴RPY，就是先转X，再转Y，再转Z
// Ref: https://stackoverflow.com/questions/54125208/eigen-eulerangles-returns-incorrect-values
// euler(0)是z轴的值
// euler(1)是y轴的值
// euler(2)是x轴的值
Eigen::Vector3d euler(1.49533, 0.32598, 0.82495);
// euler_tran(0)是x轴的值
// euler_tran(1)是y轴的值
// euler_tran(2)是z轴的值

```

```

Eigen::Vector3d euler_tran(0.82495, 0.32598, 1.49533);
// 注意区分以下代码中的euler和euler_tran

// clang-format on

// template <typename Derived>
// void Assert(const Eigen::MatrixBase<Derived>& matrix1,const Eigen::MatrixBase<Derived>&
matrix2) {
//  assert(matrix1.isApprox(matrix2,1e-8));
// }

#define ASSERT(matrix1, matrix2) assert(matrix1.isApprox(matrix2, 1e-4));

int main(int argc, char* argv[]) {
    // clang-format off
    gDebug(quaternion);
    gDebug(quaternion.coeffs().transpose());
    gDebug(rotation_matrix);
    gDebug(axis_angle.matrix());
    gDebug(euler.transpose());
    gDebug(euler_tran.transpose());
    gDebug(rotation_matrix.eulerAngles(2,1,0));

    ASSERT(rotation_matrix,quaternion.matrix());

    {
        gDebugCol2() << G_SPLIT_LINE;
        Eigen::Matrix3d rotation_matrix1 = Eigen::AxisAngleToRotationMatrix(axis, angle);
        Eigen::Matrix3d rotation_matrix2 = Eigen::AxisAngleToRotationMatrix(axis_angle);
        gDebug(rotation_matrix1);
        gDebug(rotation_matrix2);
        ASSERT(rotation_matrix,rotation_matrix1);
        ASSERT(rotation_matrix,rotation_matrix2);

        Eigen::Quaterniond quaternion1 = Eigen::AxisAngleToQuaternion(axis, angle);
        Eigen::Quaterniond quaternion2 = Eigen::AxisAngleToQuaternion(axis_angle);
        gDebug(quaternion1);
        gDebug(quaternion2);
        ASSERT(quaternion.coeffs(),quaternion1.coeffs());
        ASSERT(quaternion.coeffs(),quaternion2.coeffs());

        Eigen::Vector3d euler1 = Eigen::AxisAngleToEuler(axis, angle);
        Eigen::Vector3d euler2 = Eigen::AxisAngleToEuler(axis_angle);
        gDebug(euler1);
    }
}

```

```

gDebug(euler2);
ASSERT(euler,euler1);
ASSERT(euler,euler2);
}
{
gDebugCol2() << G_SPLIT_LINE;
Eigen::AngleAxisd axis_angle1 = Eigen::RotationMatrixToAxisAngle(rotation_matrix);
gDebug(axis_angle1.matrix());
ASSERT(axis_angle.matrix(),axis_angle1.matrix());

Eigen::Quaterniond quaternion1 = Eigen::RotationMatrixToQuaternion(rotation_matrix);
gDebug(quaternion1);
ASSERT(quaternion.coeffs(),quaternion1.coeffs());

Eigen::Vector3d euler1 = Eigen::RotationMatrixToEuler(rotation_matrix);
gDebug(euler1);
ASSERT(euler,euler1);
}
{
gDebugCol2() << G_SPLIT_LINE;
Eigen::AngleAxisd axis_angle1 = Eigen::QuaternionToAxisAngle(quaternion);
gDebug(axis_angle1.matrix());
ASSERT(axis_angle.matrix(),axis_angle1.matrix());

Eigen::Matrix3d rotation_matrix1 = Eigen::QuaternionToRotationMatrix(quaternion);
gDebug(rotation_matrix1);
ASSERT(rotation_matrix,rotation_matrix1);

Eigen::Vector3d euler1 = Eigen::QuaternionToEuler(quaternion);
gDebug(euler1);
ASSERT(euler,euler1);
}
{
gDebugCol2() << G_SPLIT_LINE;
Eigen::AngleAxisd axis_angle1 = Eigen::EulerToAxisAngle(euler_tran);
gDebug(axis_angle1.matrix());
ASSERT(axis_angle.matrix(),axis_angle1.matrix());

Eigen::Matrix3d rotation_matrix1 = Eigen::EulerToRotationMatrix(euler_tran);
gDebug(rotation_matrix1);
ASSERT(rotation_matrix,rotation_matrix1);

Eigen::Quaterniond quaternion1 = Eigen::EulerToQuaternion(euler_tran);
gDebug(quaternion1);
ASSERT(quaternion.coeffs(),quaternion1.coeffs());
}

```

```

}

{
    gDebugCol4() << G_SPLIT_LINE;
    Eigen::Matrix3d rotation_matrix1 = gxt::AxisAngleToRotationMatrix(axis, angle);
    Eigen::Matrix3d rotation_matrix2 = gxt::AxisAngleToRotationMatrix(axis_angle);
    gDebug(rotation_matrix1);
    gDebug(rotation_matrix2);
    ASSERT(rotation_matrix,rotation_matrix1);
    ASSERT(rotation_matrix,rotation_matrix2);

    Eigen::Quaterniond quaternion1 = gxt::AxisAngleToQuaternion(axis, angle);
    Eigen::Quaterniond quaternion2 = gxt::AxisAngleToQuaternion(axis_angle);
    gDebug(quaternion1);
    gDebug(quaternion2);
    ASSERT(quaternion.coeffs(),quaternion1.coeffs());
    ASSERT(quaternion.coeffs(),quaternion2.coeffs());

    Eigen::Vector3d euler1 = gxt::AxisAngleToEuler(axis, angle);
    Eigen::Vector3d euler2 = gxt::AxisAngleToEuler(axis_angle);
    gDebug(euler1);
    gDebug(euler2);
    ASSERT(euler_tran,euler1);
    ASSERT(euler_tran,euler2);
}
{
    gDebugCol4() << G_SPLIT_LINE;
    Eigen::AngleAxisd axis_angle1 = gxt::RotationMatrixToAxisAngle(rotation_matrix);
    gDebug(axis_angle1.matrix());
    ASSERT(axis_angle.matrix(),axis_angle1.matrix());

    Eigen::Quaterniond quaternion1 = gxt::RotationMatrixToQuaternion(rotation_matrix);
    gDebug(quaternion1);
    ASSERT(quaternion.coeffs(),quaternion1.coeffs());

    Eigen::Vector3d euler1 = gxt::RotationMatrixToEuler(rotation_matrix);
    gDebug(euler1);
    ASSERT(euler_tran,euler1);
}
{
    gDebugCol4() << G_SPLIT_LINE;
    Eigen::AngleAxisd axis_angle1 = gxt::QuaternionToAxisAngle(quaternion);
    gDebug(axis_angle1.matrix());
    ASSERT(axis_angle.matrix(),axis_angle1.matrix());
}

```

```

Eigen::Matrix3d rotation_matrix1 = gxt::QuaternionToRotationMatrix(quaternion);
gDebug(rotation_matrix1);
ASSERT(rotation_matrix,rotation_matrix1);

Eigen::Vector3d euler1 = gxt::QuaternionToEuler(quaternion);
gDebug(euler1);
ASSERT(euler_tran,euler1);
}
{
gDebugCol4() << G_SPLIT_LINE;
Eigen::AngleAxisd axis_angle1 = gxt::EulerToAxisAngle(euler_tran);
gDebug(axis_angle1.matrix());
ASSERT(axis_angle.matrix(),axis_angle1.matrix());

Eigen::Matrix3d rotation_matrix1 = gxt::EulerToRotationMatrix(euler_tran);
gDebug(rotation_matrix1);
ASSERT(rotation_matrix,rotation_matrix1);

Eigen::Quaterniond quaternion1 = gxt::EulerToQuaternion(euler_tran);
gDebug(quaternion1);
ASSERT(quaternion.coeffs(),quaternion1.coeffs());
}

// clang-format on
return 0;
}

```